

- 草稿树

在推测性采样中，草稿树是一种数据结构，用于存储和组织在生成过程中临时创建的候选token。

- 动态调整

与传统的静态草稿树（其结构在所有上下文中都固定不变）不同，动态草稿树根据当前的上下文信息和草稿token的预期接受率来调整其结构。

工作流程

- 扩展阶段（Expansion Phase）

EAGLE-2选择当前草稿树层中最有潜力的节点（即token），并使用草稿模型来生成下一层的候选token。选择哪些节点进行扩展是基于节点的“值”（一个结合了置信度和祖先节点置信度的度量）。

- 重排阶段（Reranking Phase）

扩展阶段的目标是加深草稿树，但选择的token可能在全局上不是最优的。因此，在重排阶段，EAGLE-2重新对所有草稿token进行排序，并选择具有最高值的前m个token。这确保了所选token仍然形成一个连接的树结构。

用示例讲解下扩展阶段和重排阶段 <https://gemini.google.com/app/21d39e366127ce06>

这张幻灯片讲解的是大语言模型推理加速技术——**推测性解码 (Speculative Decoding)** 中“草稿树 (Draft Tree)”的构建过程。

为了让你更直观地理解，我们假设大模型当前已经生成了一句话作为根节点：“我周末喜欢去”，接下来由草稿模型（一个小而快的模型）来猜测后续的词 (token)。

我们设定这棵树最终只保留 $m=3$ 个节点。

1. 扩展阶段 (Expansion Phase)

核心目标：从当前层中挑出有潜力的词，继续往下猜，把树“挖深”。

- 第1层预测：草稿模型基于“我周末喜欢去”，给出了三个猜测，并附带了它们的“潜力值” (Value)：
 - 节点 A：“逛街” (Value: 0.6)
 - 节点 B：“公园” (Value: 0.8)
 - 节点 C：“加班” (Value: 0.1)
- 执行扩展：算法认为 A 和 B 的潜力值较高，于是决定基于它们继续往下生成第2层：
 - 基于 A (逛街)，草稿模型猜下一个词是 A1：“买” (Value: 0.95) ——（草稿模型非常笃定逛街后面就是买东西）
 - 基于 B (公园)，草稿模型猜下一个词是 B1：“散步” (Value: 0.7)

到这一步，扩展阶段就帮我们生成了一批候选节点：A, B, C, A1, B1。

2. 重排阶段 (Reranking Phase) 及其红圈部分

核心目标：扩展出来的节点可能很多，我们需要从中挑出“综合价值最高”的前 m 个节点 (本例中 $m=3$)，交给大模型去最终验证。

这就是你红圈圈出那句话的关键所在：不仅要分高，还要保证能连成一棵树。

我们来看看如果不加这个限制会发生什么：

现在所有的候选节点及分数为：

- A1 (“买”): 0.95
- B (“公园”): 0.8
- B1 (“散步”): 0.7
- A (“逛街”): 0.6
- C (“加班”): 0.1

✗ 错误做法(如果不保证连接的树结构)：

如果我们只看绝对分数，挑出前3名：

第一名：A1 (“买” 0.95)

第二名：B (“公园” 0.8)

第三名：B1 (“散步” 0.7)

选出来的集合是：{A1, B, B1}。

这在逻辑上会变成灾难。当交给大模型验证时：

- “公园” -> “散步” 这条线是完整的(我周末喜欢去公园散步)。
- 但 A1 (“买”) 成了孤儿节点！因为它的父节点 A (“逛街” 0.6分)被淘汰了。大模型根本不知道这个“买”字前面接的是什么词，无法验证这半截子话。

✓ 正确做法(红圈中的逻辑：确保形成连接的树结构)：

为了解决上述“孤儿”问题，算法在挑选高分节点时，会强制要求：如果选中一个节点，它的父节点(及祖先节点)也必须被选中。

1. 系统看到分数最高的是 **A1 (“买” 0.95)**，决定选中它。
2. 强制连树规则触发：为了要 A1，系统必须把它的父节点 **A (“逛街” 0.6)** 也拉进队伍。
3. 现在队伍里有了 {A, A1}，名额只剩 1 个了(因为 $m=3$)。
4. 系统在剩下的节点中挑分数最高的，也就是 **B (“公园” 0.8)**。
5. 最终选出的 3 个节点是：{A, A1, B}。

最终交给大模型验证的结构是一棵完整的、连接的树：

- 路径1：我周末喜欢去 -> 逛街 (A) -> 买 (A1)
- 路径2：我周末喜欢去 -> 公园 (B)

总结

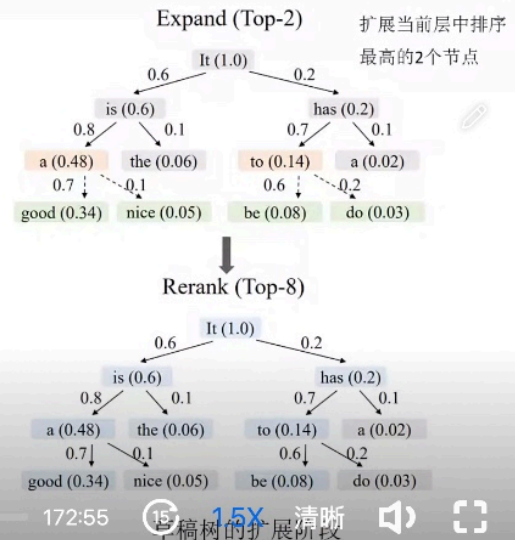
红圈里的话意思是：在重排挑选 Top-m 的 token 时，不能“唯分数论”导致树枝断裂。高分叶子节点要想被选中，必须“连根拔起”(带上它的父节点)，以此确保提交给大模型的是一条条连贯的上下文路径，而不是一堆毫无关联的散落单词。

• 扩展阶段

得益于Tree Attention，Draft Model可以同时输入当前层的所有token，并在单次前向传递中计算下一个token的概率，从而扩展当前层的所有token。然而，一次性输入太多token会减慢Draft Model的前向传递速度，而且Draft Tree每一层的token数量呈指数级增长。因此，我们需要有选择性地扩展Draft Tree。

作者从当前层中选择具有最高全局接受概率的前k个token进行扩展。在推测性采样中，如果拒绝了一个草稿token，就会导致丢弃所有后续token；一个token只有在所有前缀都被接受时才最终被接受。Token t_i 的全局接受率是Draft Tree中从根节点到 t_i 节点路径上所有token接受率的乘积。我们将其定义为 V_i ：

$$V_i = \prod_{t_j \in \text{Path}(\text{root}, t_i)} p_j \approx \prod_{t_j \in \text{Path}(\text{root}, t_i)} c_j,$$



分析一下图中右边的2个树状结构，**expand**和**rerank**，拆解一下详细步骤帮助我来理解

这张图非常清晰地展示了草稿树(Draft Tree)在生成过程中，为了平衡“算力消耗”和“猜测准确率”而采取的两个关键操作：有选择的扩展(**Expand**)和全局重排剪枝(**Rerank**)。

我们以根节点 **It (1.0)** 作为起点，一步步拆解右侧的两个树状结构。

第一步：扩展阶段 —— **Expand (Top-2)**

目标：决定草稿树中，哪些节点有资格继续往下“生长”。

如果让每一个词都继续往下猜，树的节点会呈指数级爆炸。因此，算法只在当前层挑选最有潜力的前 k 个节点往下算(图中 $k=2$)。

详细拆解(看图的上方)：

1. 当前层状态(第2层)：

在 **It -> is** 和 **It -> has** 之后，草稿模型预测出了4个候选节点。每个节点都有一个全局接受率(值, **Value**)，计算方式是路径上所有概率的乘积：

- **a** 的值： $0.6 \times 0.8 = 0.48$
- **to** 的值： $0.2 \times 0.7 = 0.14$
- **the** 的值： $0.6 \times 0.1 = 0.06$
- **a** (has下面那个) 的值： $0.2 \times 0.1 = 0.02$

2. 执行 **Top-2** 选择：

在这一层中，挑选出全局值最高的 2 个 节点：

- 第一名: **a (0.48)**
 - 第二名: **to (0.14)**
3. 生成下一层(扩展):
- 草稿模型只针对选中的这两个节点继续往后预测:
- 基于 **a (0.48)**, 预测出了 **good (0.34)** 和 **nice (0.05)**。
 - 基于 **to (0.14)**, 预测出了 **be (0.08)** 和 **do (0.03)**。
 - (注:落选的 **the (0.06)** 和 **a (0.02)** 就停止生长了, 没有向下的箭头。)

第二步: 重排阶段 —— Rerank (Top-8)

目标: 在树生长完毕后, 从所有生成的节点中, 挑出最有价值的 **\$m\$** 个节点(图中 **\$m=8\$**)交给大模型去验证。

经过上面的扩展, 我们现在一共有 10 个候选草稿节点(不含根节点 It)。但我们设定的预算是最多提交 8 个节点, 所以必须“剪掉”2 个。

详细拆解(看图的下方):

1. 全局盘点候选人:
现在树里有 10 个节点, 对应 10 个不同的全局值:
0.6, 0.48, 0.34, 0.2, 0.14, 0.08, 0.06, 0.05, 0.03, 0.02。
2. 执行剪枝(看图中蓝灰色的底色块):
根据图中带有蓝色背景的高亮块, 算法保留了以下 8 个节点:
 - **is (0.6)**
 - **has (0.2)**
 - **a (0.48)**
 - **to (0.14)**
 - **be (0.08)**
 - **the (0.06)**
 - **good (0.34)**
 - **a (0.02)**
3. 谁被淘汰了?
图中没有蓝色背景的两个节点是最底层的 **nice (0.05)** 和 **do (0.03)**。它们被作为多余的侧枝被剪掉了。

联系上一条回答中的知识点:

你可能会疑惑, 为什么全局值为 0.05 的 **nice** 被剪掉了, 而值为 0.02 的 **has -> a** 却被保留了?

这就是上一次提到的**“必须保持连接的树结构”在起作用, 或者说是算法为了保持一定程度的多样性(树的宽度)**所做的妥协。

如果一味地只取绝对分数最高的节点, 整棵树可能会变成一条极深但单一的单行道。通过 Rerank 机制, 算法切掉了延伸出来但胜率极低的末端枝桠(nice 和 do), 保留了上层的短枝(has -> a),

从而确保最终交给大模型的 8 个 token 既有高分的深度探索, 又具备连贯的树状分叉结构。